RoBERTa

[RoBERTa](https://huggingface.co/papers/1907.11692) improves BERT with new pretraining objectives, demonstrating [BERT](./bert) was undertrained and training design is important. The pretraining objectives include dynamic masking, sentence packing, larger batches and a byte-level BPE tokenizer.

You can find all the original RoBERTa checkpoints under the [Facebook AI](https://huggingface.co/FacebookAI) organization.

> [!TIP]
> Click on the RoBERTa models in the right sidebar for more examples of how to apply RoBERTa to different language tasks.

The example below demonstrates how to predict the `<mask>` token with [Pipeline](/docs/transformers/v5.14.0/en/main_classes/pipelines#transformers.Pipeline), [AutoModel](/docs/transformers/v5.14.0/en/model_doc/auto#transformers.AutoModel), and from the command line.

```python
from transformers import pipeline

pipeline = pipeline(
    task="fill-mask",
    model="FacebookAI/roberta-base",
    device=0
)
pipeline("Plants create <mask> through a process known as photosynthesis.")
```

```python
import torch

from transformers import AutoModelForMaskedLM, AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained(
    "FacebookAI/roberta-base",
)
model = AutoModelForMaskedLM.from_pretrained(
    "FacebookAI/roberta-base",
    device_map="auto",
    attn_implementation="sdpa"
)
inputs = tokenizer("Plants create <mask> through a process known as photosynthesis.", return_tensors="pt").to(model.device)

with torch.no_grad():
```

```
    outputs = model(**inputs)
    predictions = outputs.logits

masked_index = torch.where(inputs['input_ids'] == tokenizer.mask_token_id)[1]
predicted_token_id = predictions[0, masked_index].argmax(dim=-1)
predicted_token = tokenizer.decode(predicted_token_id)

print(f"The predicted token is: {predicted_token}")
```

Notes

- RoBERTa doesn't have `token_type_ids` so you don't need to indicate which token belongs to which segment. Separate your segments with the separation token `tokenizer.sep_token` or `</s>`.

RobertaConfig[[transformers.RobertaConfig]]

- **vocab_size** (`int`, *optional*, defaults to `50265`) --
  Vocabulary size of the model. Defines the number of different tokens that can be represented by the `input_ids`.
- **hidden_size** (`int`, *optional*, defaults to `768`) --
  Dimension of the hidden representations.
- **num_hidden_layers** (`int`, *optional*, defaults to `12`) --
  Number of hidden layers in the Transformer decoder.
- **num_attention_heads** (`int`, *optional*, defaults to `12`) --
  Number of attention heads for each attention layer in the Transformer decoder.
- **intermediate_size** (`int`, *optional*, defaults to `3072`) --
  Dimension of the MLP representations.
- **hidden_act** (`str`, *optional*, defaults to `gelu`) --
  The non-linear activation function (function or string) in the decoder. For example, `"gelu"`,
  `"relu"`, `"silu"`, etc.
- **hidden_dropout_prob** (`Union[float, int]`, *optional*, defaults to `0.1`) --
  The dropout probability for all fully connected layers in the embeddings, encoder, and pooler.
- **attention_probs_dropout_prob** (`Union[float, int]`, *optional*, defaults to `0.1`) --
  The dropout ratio for the attention probabilities.
- **max_position_embeddings** (`int`, *optional*, defaults to `512`) --
  The maximum sequence length that this model might ever be used with.
- **type_vocab_size** (`int`, *optional*, defaults to `2`) --
  The vocabulary size of the `token_type_ids`.
- **initializer_range** (`float`, *optional*, defaults to `0.02`) --
  The standard deviation of the truncated_normal_initializer for initializing all weight matrices.
- **layer_norm_eps** (`float`, *optional*, defaults to `1e-12`) --
  The epsilon used by the layer normalization layers.

- **pad_token_id** (`int`, *optional*, defaults to `1`) --
  Token id used for padding in the vocabulary.
- **bos_token_id** (`int`, *optional*, defaults to `0`) --
  Token id used for beginning-of-stream in the vocabulary.
- **eos_token_id** (`Union[int, list[int]]`, *optional*, defaults to `2`) --
  Token id used for end-of-stream in the vocabulary.
- **use_cache** (`bool`, *optional*, defaults to `True`) --
  Whether or not the model should return the last key/values attentions (not used by all models). Only
  relevant if `config.is_decoder=True` or when the model is a decoder-only generative model.
- **classifier_dropout** (`Union[float, int]`, *optional*) --
  The dropout ratio for classifier.
- **is_decoder** (`bool`, *optional*, defaults to `False`) --
  Whether the model is used as a decoder or not. If `False`, the model is used as an encoder.
- **add_cross_attention** (`bool`, *optional*, defaults to `False`) --
  Whether cross-attention layers should be added to the model.
- **tie_word_embeddings** (`bool`, *optional*, defaults to `True`) --
  Whether to tie weight embeddings according to model's `tied_weights_keys` mapping.

This is the configuration class to store the configuration of a RobertaModel. It is used to instantiate a Roberta
model according to the specified arguments, defining the model architecture. Instantiating a configuration with the
defaults will yield a similar configuration to that of the [FacebookAI/roberta-base](https://huggingface.co/FacebookAI/roberta-base)

Configuration objects inherit from [PreTrainedConfig]
(/docs/transformers/v5.14.0/en/main_classes/configuration#transformers.PreTrainedConfig) and can be used to control the model outputs. Read the
documentation from [PreTrainedConfig]
(/docs/transformers/v5.14.0/en/main_classes/configuration#transformers.PreTrainedConfig) for more information.

Examples:

```python
>>> from transformers import RobertaConfig, RobertaModel

>>> # Initializing a RoBERTa configuration
>>> configuration = RobertaConfig()

>>> # Initializing a model (with random weights) from the configuration
>>> model = RobertaModel(configuration)

>>> # Accessing the model configuration
```

```
>>> configuration = model.config
```

RobertaTokenizer[[transformers.RobertaTokenizer]]

'"}, {"name": "eos_token", "val": ": str = '"}, {"name": "sep_token", "val": ": str = '"}, {"name": "cls_token", "val": ": str = '"}, {"name": "unk_token", "val": ": str = '"}, {"name": "pad_token", "val": ": str = '"}, {"name": "mask_token", "val": ": str = '"}, {"name": "add_prefix_space", "val": ": bool = False"}, {"name": "trim_offsets", "val": ": bool = True"}, {"name": "**kwargs", "val": ""}]}>
- **vocab** (`str`, `dict` or `list`, *optional*) --
  Custom vocabulary dictionary. If not provided, vocabulary is loaded from vocab_file.
- **merges** (`str` or `list`, *optional*) --
  Custom merges list. If not provided, merges are loaded from merges_file.
- **errors** (`str`, *optional*, defaults to `"replace"`) --
  Paradigm to follow when decoding bytes to UTF-8. See
  [bytes.decode](https://docs.python.org/3/library/stdtypes.html#bytes.decode) for more information.
- **bos_token** (`str`, *optional*, defaults to `"<s>"`) --
  The beginning of sequence token that was used during pretraining. Can be used a sequence classifier token.

  When building a sequence using special tokens, this is not the token that is used for the beginning of
  sequence. The token used is the `cls_token`.

- **eos_token** (`str`, *optional*, defaults to `"</s>"`) --
  The end of sequence token.

  When building a sequence using special tokens, this is not the token that is used for the end of sequence.
  The token used is the `sep_token`.

- **sep_token** (`str`, *optional*, defaults to `"</s>"`) --
  The separator token, which is used when building a sequence from multiple sequences, e.g. two sequences for
  sequence classification or for a text and a question for question answering. It is also used as the last

  token of a sequence built with special tokens.
- **cls_token** (`str`, *optional*, defaults to `"<s>"`) --
  The classifier token which is used when doing sequence classification (classification of the whole sequence
  instead of per-token classification). It is the first token of the sequence when built with special tokens.
- **unk_token** (`str`, *optional*, defaults to `"<unk>"`) --
  The unknown token. A token that is not in the vocabulary cannot be converted to an ID and is set to be this
  token instead.
- **pad_token** (`str`, *optional*, defaults to `"<pad>"`) --
  The token used for padding, for example when batching sequences of different lengths.
- **mask_token** (`str`, *optional*, defaults to `"<mask>"`) --
  The token used for masking values. This is the token used when training this model with masked language
  modeling. This is the token which the model will try to predict.
- **add_prefix_space** (`bool`, *optional*, defaults to `False`) --
  Whether or not to add an initial space to the input. This allows to treat the leading word just as any
  other word. (RoBERTa tokenizer detect beginning of words by the preceding space).
- **trim_offsets** (`bool`, *optional*, defaults to `True`) --
  Whether the post processing step should trim offsets to avoid including whitespaces.

Construct a RoBERTa tokenizer (backed by HuggingFace's tokenizers library). Based on Byte-Pair-Encoding.

This tokenizer has been trained to treat spaces like parts of the tokens (a bit like sentencepiece) so a word will

be encoded differently whether it is at the beginning of the sentence (without space) or not:

```python
>>> from transformers import RobertaTokenizer

>>> tokenizer = RobertaTokenizer.from_pretrained("FacebookAI/roberta-base")
>>> tokenizer("Hello world")["input_ids"]
[0, 31414, 232, 2]

>>> tokenizer(" Hello world")["input_ids"]
[0, 20920, 232, 2]
```

You can get around that behavior by passing `add_prefix_space=True` when instantiating this tokenizer or when you

call it on some text, but since the model was not pretrained this way, it might yield a decrease in performance.

When used with `is_split_into_words=True`, this tokenizer needs to be instantiated with `add_prefix_space=True`.

This tokenizer inherits from [TokenizersBackend] (/docs/transformers/v5.14.0/en/main_classes/tokenizer#transformers.TokenizersBackend) which contains most of the main methods. Users should refer to this superclass for more information regarding those methods.

- **token_ids_0** -- List of IDs for the (possibly already formatted) sequence.
- **token_ids_1** -- Unused when `already_has_special_tokens=True`. Must be None in that case.
- **already_has_special_tokens** -- Whether the sequence is already formatted with special tokens.A list of integers in the range [0, 1]1 for a special token, 0 for a sequence token.

Retrieve sequence ids from a token list that has no special tokens added.

For fast tokenizers, data collators call this with `already_has_special_tokens=True` to build a mask over an already-formatted sequence. In that case, we compute the mask by checking membership in `all_special_ids`.

RobertaTokenizerFast[[transformers.RobertaTokenizerFast]]

'"}, {"name": "eos_token", "val": " = '"}, {"name": "sep_token", "val": " = '"}, {"name": "cls_token", "val": " = '"}, {"name": "unk_token", "val": " = '"}, {"name": "pad_token", "val": " = '"}, {"name": "mask_token", "val": " = '"}, {"name": "add_prefix_space", "val": " = False"}, {"name": "trim_offsets", "val": " = True"}, {"name": "**kwargs", "val": ""}]}>
- **vocab_file** (`str`) --
  Path to the vocabulary file.
- **merges_file** (`str`) --
  Path to the merges file.
- **errors** (`str`, *optional*, defaults to `"replace"`) --
  Paradigm to follow when decoding bytes to UTF-8. See
  [bytes.decode](https://docs.python.org/3/library/stdtypes.html#bytes.decode) for more information.
- **bos_token** (`str`, *optional*, defaults to `"<s>"`) --
  The beginning of sequence token that was used during pretraining. Can be used a sequence classifier token.

  When building a sequence using special tokens, this is not the token that is used for the beginning of

  sequence. The token used is the `cls_token`.

- **eos_token** (`str`, *optional*, defaults to `"</s>"`) --
  The end of sequence token.

  When building a sequence using special tokens, this is not the token that is used for the end of sequence.
  The token used is the `sep_token`.

- **sep_token** (`str`, *optional*, defaults to `"</s>"`) --
  The separator token, which is used when building a sequence from multiple sequences, e.g. two sequences for
  sequence classification or for a text and a question for question answering. It is also used as the last
  token of a sequence built with special tokens.
- **cls_token** (`str`, *optional*, defaults to `"<s>"`) --
  The classifier token which is used when doing sequence classification (classification of the whole sequence
  instead of per-token classification). It is the first token of the sequence when built with special tokens.
- **unk_token** (`str`, *optional*, defaults to `"<unk>"`) --
  The unknown token. A token that is not in the vocabulary cannot be converted to an ID and is set to be this
  token instead.
- **pad_token** (`str`, *optional*, defaults to `"<pad>"`) --
  The token used for padding, for example when batching sequences of different lengths.
- **mask_token** (`str`, *optional*, defaults to `"<mask>"`) --
  The token used for masking values. This is the token used when training this model with masked language
  modeling. This is the token which the model will try to predict.
- **add_prefix_space** (`bool`, *optional*, defaults to `False`) --
  Whether or not to add an initial space to the input. This allows to treat the leading word just as any
  other word. (RoBERTa tokenizer detect beginning of words by the preceding space).
- **trim_offsets** (`bool`, *optional*, defaults to `True`) --
  Whether the post processing step should trim offsets to avoid including whitespaces.

Construct a "fast" RoBERTa tokenizer (backed by HuggingFace's *tokenizers* library), derived from the GPT-2

tokenizer, using byte-level Byte-Pair-Encoding.

This tokenizer has been trained to treat spaces like parts of the tokens (a bit like sentencepiece) so a word will

be encoded differently whether it is at the beginning of the sentence (without space) or not:

```python
>>> from transformers import RobertaTokenizerFast

>>> tokenizer = RobertaTokenizerFast.from_pretrained("FacebookAI/roberta-base")
>>> tokenizer("Hello world")["input_ids"]
[0, 31414, 232, 2]

>>> tokenizer(" Hello world")["input_ids"]
[0, 20920, 232, 2]
```

You can get around that behavior by passing `add_prefix_space=True` when instantiating this tokenizer or when you
call it on some text, but since the model was not pretrained this way, it might yield a decrease in performance.

When used with `is_split_into_words=True`, this tokenizer needs to be instantiated with `add_prefix_space=True`.

This tokenizer inherits from [PreTrainedTokenizerFast](/docs/transformers/v5.14.0/en/main_classes/tokenizer#transformers.TokenizersBackend) which contains most of the main methods. Users should
refer to this superclass for more information regarding those methods.

- **token_ids_0** (`list[int]`) --
  List of IDs.
- **token_ids_1** (`list[int]`, *optional*) --
  Optional second list of IDs for sequence pairs.`list[int]`List of zeros.

Create a mask from the two sequences passed to be used in a sequence-pair classification task. RoBERTa does not
make use of token type ids, therefore a list of zeros is returned.

RobertaModel[[transformers.RobertaModel]]

- **config** ([RobertaModel](/docs/transformers/v5.14.0/en/model_doc/roberta#transformers.RobertaModel)) --
  Model configuration class with all the parameters of the model. Initializing with a config file does not
  load the weights associated with the model, only the configuration. Check out

the
  [from_pretrained()]
(/docs/transformers/v5.14.0/en/main_classes/model#transformers.PreTrainedModel.fro
m_pretrained) method to load the model weights.
- **add_pooling_layer** (`bool`, *optional*, defaults to `True`) --
  Whether to add a pooling layer

The model can behave as an encoder (with only self-attention) as well as a
decoder, in which case a layer of
cross-attention is added between the self-attention layers, following the
architecture described in [Attention is
all you need](https://huggingface.co/papers/1706.03762) by Ashish Vaswani, Noam
Shazeer, Niki Parmar, Jakob Uszkoreit,
Llion Jones, Aidan N. Gomez, Lukasz Kaiser and Illia Polosukhin.

To behave as an decoder the model needs to be initialized with the `is_decoder`
argument of the configuration set
to `True`. To be used in a Seq2Seq model, the model needs to initialized with both
`is_decoder` argument and
`add_cross_attention` set to `True`; an `encoder_hidden_states` is then expected
as an input to the forward pass.

This model inherits from [PreTrainedModel]
(/docs/transformers/v5.14.0/en/main_classes/model#transformers.PreTrainedModel).
Check the superclass documentation for the generic methods the
library implements for all its model (such as downloading or saving, resizing the
input embeddings, pruning heads
etc.)

This model is also a PyTorch [torch.nn.Module]
(https://pytorch.org/docs/stable/nn.html#torch.nn.Module) subclass.
Use it as a regular PyTorch Module and refer to the PyTorch documentation for all
matter related to general usage
and behavior.

- **input_ids** (`torch.Tensor` of shape `(batch_size, sequence_length)`,
optional) --
  Indices of input sequence tokens in the vocabulary. Padding will be ignored by
default.

  Indices can be obtained using [AutoTokenizer]
(/docs/transformers/v5.14.0/en/model_doc/auto#transformers.AutoTokenizer). See
[PreTrainedTokenizer.encode()]
(/docs/transformers/v5.14.0/en/internal/tokenization_utils#transformers.PreTrained
TokenizerBase.encode) and
  [PreTrainedTokenizer.__call__()]
(/docs/transformers/v5.14.0/en/internal/tokenization_utils#transformers.PreTrained
TokenizerBase.__call__) for details.

  [What are input IDs?](../glossary#input-ids)
- **attention_mask** (`torch.Tensor` of shape `(batch_size, sequence_length)`, *optional*) --
  Mask to avoid performing attention on padding token indices. Mask values selected in `[0, 1]`:

  - 1 for tokens that are **not masked**,
  - 0 for tokens that are **masked**.

  [What are attention masks?](../glossary#attention-mask)
- **token_type_ids** (`torch.Tensor` of shape `(batch_size, sequence_length)`, *optional*) --
  Segment token indices to indicate first and second portions of the inputs. Indices are selected in `[0, 1]`:

  - 0 corresponds to a *sentence A* token,
  - 1 corresponds to a *sentence B* token.

  [What are token type IDs?](../glossary#token-type-ids)
- **position_ids** (`torch.Tensor` of shape `(batch_size, sequence_length)`, *optional*) --
  Indices of positions of each input sequence tokens in the position embeddings. Selected in the range `[0, config.n_positions - 1]`.

  [What are position IDs?](../glossary#position-ids)
- **inputs_embeds** (`torch.Tensor` of shape `(batch_size, sequence_length, hidden_size)`, *optional*) --
  Optionally, instead of passing `input_ids` you can choose to directly pass an embedded representation. This
  is useful if you want more control over how to convert `input_ids` indices into associated vectors than the
  model's internal embedding lookup matrix.
- **encoder_hidden_states** (`torch.Tensor` of shape `(batch_size, sequence_length, hidden_size)`, *optional*) --
  Sequence of hidden-states at the output of the last layer of the encoder. Used in the cross-attention
  if the model is configured as a decoder.
- **encoder_attention_mask** (`torch.Tensor` of shape `(batch_size, sequence_length)`, *optional*) --
  Mask to avoid performing attention on the padding token indices of the encoder input. This mask is used in
  the cross-attention if the model is configured as a decoder. Mask values selected in `[0, 1]`:

  - 1 for tokens that are **not masked**,
  - 0 for tokens that are **masked**.
- **past_key_values** (`~cache_utils.Cache`, *optional*) --

  Pre-computed hidden-states (key and values in the self-attention blocks and in the cross-attention
  blocks) that can be used to speed up sequential decoding. This typically consists in the `past_key_values`
  returned by the model at a previous stage of decoding, when `use_cache=True` or `config.use_cache=True`.

  Only [Cache] (/docs/transformers/v5.14.0/en/internal/generation_utils#transformers.Cache) instance is allowed as input, see our [kv cache guide] (https://huggingface.co/docs/transformers/en/kv_cache).
  If no `past_key_values` are passed, [DynamicCache] (/docs/transformers/v5.14.0/en/internal/generation_utils#transformers.DynamicCache) will be initialized by default.

  The model will output the same cache format that is fed as input.

  If `past_key_values` are used, the user is expected to input only unprocessed `input_ids` (those that don't
  have their past key value states given to this model) of shape `(batch_size, unprocessed_length)` instead of all `input_ids`
  of shape `(batch_size, sequence_length)`.
- **use_cache** (`bool`, *optional*) --
  If set to `True`, `past_key_values` key value states are returned and can be used to speed up decoding (see
  `past_key_values`).[BaseModelOutputWithPoolingAndCrossAttentions] (/docs/transformers/v5.14.0/en/main_classes/output#transformers.modeling_outputs.BaseModelOutputWithPoolingAndCrossAttentions) or `tuple(torch.FloatTensor)`A [BaseModelOutputWithPoolingAndCrossAttentions] (/docs/transformers/v5.14.0/en/main_classes/output#transformers.modeling_outputs.BaseModelOutputWithPoolingAndCrossAttentions) or a tuple of `torch.FloatTensor` (if `return_dict=False` is passed or when `config.return_dict=False`) comprising various elements depending on the configuration ([RobertaConfig] (/docs/transformers/v5.14.0/en/model_doc/roberta#transformers.RobertaConfig)) and inputs.
The [RobertaModel] (/docs/transformers/v5.14.0/en/model_doc/roberta#transformers.RobertaModel) forward method, overrides the `__call__` special method.

Although the recipe for forward pass needs to be defined within this function, one should call the `Module`
instance afterwards instead of this since the former takes care of running the pre and post processing steps while
the latter silently ignores them.

- **last_hidden_state** (`torch.FloatTensor` of shape `(batch_size, sequence_length, hidden_size)`) -- Sequence of hidden-states at the output of the

last layer of the model.
- **pooler_output** (`torch.FloatTensor` of shape `(batch_size, hidden_size)`) -- Last layer hidden-state of the first token of the sequence (classification token) after further processing
  through the layers used for the auxiliary pretraining task. E.g. for BERT-family of models, this returns
  the classification token after processing through a linear layer and a tanh activation function. The linear
  layer weights are trained from the next sentence prediction (classification) objective during pretraining.
- **hidden_states** (`tuple(torch.FloatTensor)`, *optional*, returned when `output_hidden_states=True` is passed or when `config.output_hidden_states=True`) -- Tuple of `torch.FloatTensor` (one for the output of the embeddings, if the model has an embedding layer, +
  one for the output of each layer) of shape `(batch_size, sequence_length, hidden_size)`.

  Hidden-states of the model at the output of each layer plus the optional initial embedding outputs.
- **attentions** (`tuple(torch.FloatTensor)`, *optional*, returned when `output_attentions=True` is passed or when `config.output_attentions=True`) -- Tuple of `torch.FloatTensor` (one for each layer) of shape `(batch_size, num_heads, sequence_length,
  sequence_length)`.

  Attentions weights after the attention softmax, used to compute the weighted average in the self-attention
  heads.
- **cross_attentions** (`tuple(torch.FloatTensor)`, *optional*, returned when `output_attentions=True` and `config.add_cross_attention=True` is passed or when `config.output_attentions=True`) -- Tuple of `torch.FloatTensor` (one for each layer) of shape `(batch_size, num_heads, sequence_length,
  sequence_length)`.

  Attentions weights of the decoder's cross-attention layer, after the attention softmax, used to compute the
  weighted average in the cross-attention heads.
- **past_key_values** (`Cache`, *optional*, returned when `use_cache=True` is passed or when `config.use_cache=True`) -- It is a [Cache] (/docs/transformers/v5.14.0/en/internal/generation_utils#transformers.Cache) instance. For more details, see our [kv cache guide] (https://huggingface.co/docs/transformers/en/kv_cache).

  Contains pre-computed hidden-states (key and values in the self-attention blocks and optionally if
  `config.is_encoder_decoder=True` in the cross-attention blocks) that can be used (see `past_key_values`
  input) to speed up sequential decoding.

RobertaForCausalLM[[transformers.RobertaForCausalLM]]

- **config** ([RobertaForCausalLM]
(/docs/transformers/v5.14.0/en/model_doc/roberta#transformers.RobertaForCausalLM))
--
  Model configuration class with all the parameters of the model. Initializing
with a config file does not
  load the weights associated with the model, only the configuration. Check out
the
  [from_pretrained()]
(/docs/transformers/v5.14.0/en/main_classes/model#transformers.PreTrainedModel.fro
m_pretrained) method to load the model weights.

RoBERTa Model with a `language modeling` head on top for CLM fine-tuning.

This model inherits from [PreTrainedModel]
(/docs/transformers/v5.14.0/en/main_classes/model#transformers.PreTrainedModel).
Check the superclass documentation for the generic methods the
library implements for all its model (such as downloading or saving, resizing the
input embeddings, pruning heads
etc.)

This model is also a PyTorch [torch.nn.Module]
(https://pytorch.org/docs/stable/nn.html#torch.nn.Module) subclass.
Use it as a regular PyTorch Module and refer to the PyTorch documentation for all
matter related to general usage
and behavior.

- **input_ids** (`torch.LongTensor` of shape `(batch_size, sequence_length)`,
optional) --
  Indices of input sequence tokens in the vocabulary. Padding will be ignored by
default.

  Indices can be obtained using [AutoTokenizer]
(/docs/transformers/v5.14.0/en/model_doc/auto#transformers.AutoTokenizer). See
[PreTrainedTokenizer.encode()]
(/docs/transformers/v5.14.0/en/internal/tokenization_utils#transformers.PreTrained
TokenizerBase.encode) and
  [PreTrainedTokenizer.__call__()]
(/docs/transformers/v5.14.0/en/internal/tokenization_utils#transformers.PreTrained
TokenizerBase.__call__) for details.

  [What are input IDs?](../glossary#input-ids)
- **attention_mask** (`torch.FloatTensor` of shape `(batch_size,
sequence_length)`, *optional*) --
  Mask to avoid performing attention on padding token indices. Mask values
selected in `[0, 1]`:

  - 1 for tokens that are **not masked**,
  - 0 for tokens that are **masked**.

  [What are attention masks?](../glossary#attention-mask)
- **token_type_ids** (`torch.LongTensor` of shape `(batch_size, sequence_length)`, *optional*) --
  Segment token indices to indicate first and second portions of the inputs. Indices are selected in `[0,1]`:

  - 0 corresponds to a *sentence A* token,
  - 1 corresponds to a *sentence B* token.
  This parameter can only be used when the model is initialized with `type_vocab_size` parameter with value
  >= 2. All the value in this tensor should be always < type_vocab_size.

  [What are token type IDs?](../glossary#token-type-ids)
- **position_ids** (`torch.LongTensor` of shape `(batch_size, sequence_length)`, *optional*) --
  Indices of positions of each input sequence tokens in the position embeddings. Selected in the range `[0, config.n_positions - 1]`.

  [What are position IDs?](../glossary#position-ids)
- **inputs_embeds** (`torch.FloatTensor` of shape `(batch_size, sequence_length, hidden_size)`, *optional*) --
  Optionally, instead of passing `input_ids` you can choose to directly pass an embedded representation. This
  is useful if you want more control over how to convert `input_ids` indices into associated vectors than the
  model's internal embedding lookup matrix.
- **encoder_hidden_states** (`torch.FloatTensor` of shape `(batch_size, sequence_length, hidden_size)`, *optional*) --
  Sequence of hidden-states at the output of the last layer of the encoder. Used in the cross-attention
  if the model is configured as a decoder.
- **encoder_attention_mask** (`torch.FloatTensor` of shape `(batch_size, sequence_length)`, *optional*) --
  Mask to avoid performing attention on the padding token indices of the encoder input. This mask is used in
  the cross-attention if the model is configured as a decoder. Mask values selected in `[0, 1]`:

  - 1 for tokens that are **not masked**,
  - 0 for tokens that are **masked**.
- **labels** (`torch.LongTensor` of shape `(batch_size, sequence_length)`, *optional*) --
  Labels for computing the left-to-right language modeling loss (next word prediction). Indices should be in

`[-100, 0, ..., config.vocab_size]` (see `input_ids` docstring) Tokens with indices set to `-100` are
ignored (masked), the loss is only computed for the tokens with labels in `[0, ..., config.vocab_size]`
- **past_key_values** (`tuple[tuple[torch.FloatTensor]]`, *optional*) --
Pre-computed hidden-states (key and values in the self-attention blocks and in the cross-attention
blocks) that can be used to speed up sequential decoding. This typically consists in the `past_key_values`
returned by the model at a previous stage of decoding, when `use_cache=True` or `config.use_cache=True`.

Only [Cache] (/docs/transformers/v5.14.0/en/internal/generation_utils#transformers.Cache) instance is allowed as input, see our [kv cache guide] (https://huggingface.co/docs/transformers/en/kv_cache).
If no `past_key_values` are passed, [DynamicCache] (/docs/transformers/v5.14.0/en/internal/generation_utils#transformers.DynamicCache) will be initialized by default.

The model will output the same cache format that is fed as input.

If `past_key_values` are used, the user is expected to input only unprocessed `input_ids` (those that don't
have their past key value states given to this model) of shape `(batch_size, unprocessed_length)` instead of all `input_ids`
of shape `(batch_size, sequence_length)`.
- **use_cache** (`bool`, *optional*) --
If set to `True`, `past_key_values` key value states are returned and can be used to speed up decoding (see
`past_key_values`).
- **logits_to_keep** (`Union[int, torch.Tensor]`, *optional*, defaults to `0`) --
If an `int`, compute logits for the last `logits_to_keep` tokens. If `0`, calculate logits for all
`input_ids` (special case). Only last token logits are needed for generation, and calculating them only for that
token can save memory, which becomes pretty significant for long sequences or large vocabulary size.
If a `torch.Tensor`, must be 1D corresponding to the indices to keep in the sequence length dimension.
This is useful when using packed tensor format (single dimension for batch and sequence length).[CausalLMOutputWithCrossAttentions] (/docs/transformers/v5.14.0/en/main_classes/output#transformers.modeling_outputs.CausalLMOutputWithCrossAttentions) or `tuple(torch.FloatTensor)`A [CausalLMOutputWithCrossAttentions] (/docs/transformers/v5.14.0/en/main_classes/output#transformers.modeling_outputs.CausalLMOutputWithCrossAttentions) or a tuple of `torch.FloatTensor` (if `return_dict=False` is passed or when

`config.return_dict=False`) comprising various
elements depending on the configuration ([RobertaConfig]
(/docs/transformers/v5.14.0/en/model_doc/roberta#transformers.RobertaConfig)) and
inputs.
The [RobertaForCausalLM]
(/docs/transformers/v5.14.0/en/model_doc/roberta#transformers.RobertaForCausalLM)
forward method, overrides the `__call__` special method.

Although the recipe for forward pass needs to be defined within this function, one
should call the `Module`
instance afterwards instead of this since the former takes care of running the pre
and post processing steps while
the latter silently ignores them.

- **loss** (`torch.FloatTensor` of shape `(1,)`, *optional*, returned when
`labels` is provided) -- Language modeling loss (for next-token prediction).
- **logits** (`torch.FloatTensor` of shape `(batch_size, sequence_length,
config.vocab_size)`) -- Prediction scores of the language modeling head (scores
for each vocabulary token before SoftMax).
- **hidden_states** (`tuple(torch.FloatTensor)`, *optional*, returned when
`output_hidden_states=True` is passed or when `config.output_hidden_states=True`)
-- Tuple of `torch.FloatTensor` (one for the output of the embeddings, if the
model has an embedding layer, +
  one for the output of each layer) of shape `(batch_size, sequence_length,
hidden_size)`.

  Hidden-states of the model at the output of each layer plus the optional initial
embedding outputs.
- **attentions** (`tuple(torch.FloatTensor)`, *optional*, returned when
`output_attentions=True` is passed or when `config.output_attentions=True`) --
Tuple of `torch.FloatTensor` (one for each layer) of shape `(batch_size,
num_heads, sequence_length,
  sequence_length)`.

  Attentions weights after the attention softmax, used to compute the weighted
average in the self-attention
  heads.
- **cross_attentions** (`tuple(torch.FloatTensor)`, *optional*, returned when
`output_attentions=True` is passed or when `config.output_attentions=True`) --
Tuple of `torch.FloatTensor` (one for each layer) of shape `(batch_size,
num_heads, sequence_length,
  sequence_length)`.

  Cross attentions weights after the attention softmax, used to compute the
weighted average in the
  cross-attention heads.
- **past_key_values** (`Cache`, *optional*, returned when `use_cache=True` is
passed or when `config.use_cache=True`) -- It is a [Cache]

(/docs/transformers/v5.14.0/en/internal/generation_utils#transformers.Cache)
instance. For more details, see our [kv cache guide]
(https://huggingface.co/docs/transformers/en/kv_cache).

  Contains pre-computed hidden-states (key and values in the attention blocks)
that can be used (see
  `past_key_values` input) to speed up sequential decoding.

Example:

```python
>>> from transformers import AutoTokenizer, RobertaForCausalLM, AutoConfig
>>> import torch

>>> tokenizer = AutoTokenizer.from_pretrained("FacebookAI/roberta-base")
>>> config = AutoConfig.from_pretrained("FacebookAI/roberta-base")
>>> config.is_decoder = True
>>> model = RobertaForCausalLM.from_pretrained("FacebookAI/roberta-base",
config=config)

>>> inputs = tokenizer("Hello, my dog is cute", return_tensors="pt")
>>> outputs = model(**inputs)

>>> prediction_logits = outputs.logits
```

RobertaForMaskedLM[[transformers.RobertaForMaskedLM]]

- **config** ([RobertaForMaskedLM]
(/docs/transformers/v5.14.0/en/model_doc/roberta#transformers.RobertaForMaskedLM))
--
  Model configuration class with all the parameters of the model. Initializing
with a config file does not
  load the weights associated with the model, only the configuration. Check out
the
  [from_pretrained()]
(/docs/transformers/v5.14.0/en/main_classes/model#transformers.PreTrainedModel.fro
m_pretrained) method to load the model weights.

The Roberta Model with a `language modeling` head on top."

This model inherits from [PreTrainedModel]
(/docs/transformers/v5.14.0/en/main_classes/model#transformers.PreTrainedModel).
Check the superclass documentation for the generic methods the
library implements for all its model (such as downloading or saving, resizing the
input embeddings, pruning heads
etc.)

This model is also a PyTorch [torch.nn.Module]
(https://pytorch.org/docs/stable/nn.html#torch.nn.Module) subclass.
Use it as a regular PyTorch Module and refer to the PyTorch documentation for all
matter related to general usage
and behavior.

- **input_ids** (`torch.LongTensor` of shape `(batch_size, sequence_length)`,
optional) --
  Indices of input sequence tokens in the vocabulary. Padding will be ignored by
default.

  Indices can be obtained using [AutoTokenizer]
(/docs/transformers/v5.14.0/en/model_doc/auto#transformers.AutoTokenizer). See
[PreTrainedTokenizer.encode()]
(/docs/transformers/v5.14.0/en/internal/tokenization_utils#transformers.PreTrained
TokenizerBase.encode) and
  [PreTrainedTokenizer.__call__()]
(/docs/transformers/v5.14.0/en/internal/tokenization_utils#transformers.PreTrained
TokenizerBase.__call__) for details.

  [What are input IDs?](../glossary#input-ids)
- **attention_mask** (`torch.FloatTensor` of shape `(batch_size,
sequence_length)`, *optional*) --
  Mask to avoid performing attention on padding token indices. Mask values
selected in `[0, 1]`:

  - 1 for tokens that are **not masked**,
  - 0 for tokens that are **masked**.

  [What are attention masks?](../glossary#attention-mask)
- **token_type_ids** (`torch.LongTensor` of shape `(batch_size, sequence_length)`,
optional) --
  Segment token indices to indicate first and second portions of the inputs.
Indices are selected in `[0,1]`:

  - 0 corresponds to a *sentence A* token,
  - 1 corresponds to a *sentence B* token.
  This parameter can only be used when the model is initialized with
`type_vocab_size` parameter with value
  >= 2. All the value in this tensor should be always < type_vocab_size.

  [What are token type IDs?](../glossary#token-type-ids)
- **position_ids** (`torch.LongTensor` of shape `(batch_size, sequence_length)`,
optional) --
  Indices of positions of each input sequence tokens in the position embeddings.
Selected in the range `[0, config.n_positions - 1]`.

  [What are position IDs?](../glossary#position-ids)

- **inputs_embeds** (`torch.FloatTensor` of shape `(batch_size, sequence_length, hidden_size)`, *optional*) --
  Optionally, instead of passing `input_ids` you can choose to directly pass an embedded representation. This
  is useful if you want more control over how to convert `input_ids` indices into associated vectors than the
  model's internal embedding lookup matrix.
- **encoder_hidden_states** (`torch.FloatTensor` of shape `(batch_size, sequence_length, hidden_size)`, *optional*) --
  Sequence of hidden-states at the output of the last layer of the encoder. Used in the cross-attention
  if the model is configured as a decoder.
- **encoder_attention_mask** (`torch.FloatTensor` of shape `(batch_size, sequence_length)`, *optional*) --
  Mask to avoid performing attention on the padding token indices of the encoder input. This mask is used in
  the cross-attention if the model is configured as a decoder. Mask values selected in `[0, 1]`:

  - 1 for tokens that are **not masked**,
  - 0 for tokens that are **masked**.
- **labels** (`torch.LongTensor` of shape `(batch_size, sequence_length)`, *optional*) --
  Labels for computing the masked language modeling loss. Indices should be in `[-100, 0, ...,
  config.vocab_size]` (see `input_ids` docstring) Tokens with indices set to `-100` are ignored (masked), the
  loss is only computed for the tokens with labels in `[0, ..., config.vocab_size]`[MaskedLMOutput] (/docs/transformers/v5.14.0/en/main_classes/output#transformers.modeling_outputs.MaskedLMOutput) or `tuple(torch.FloatTensor)`A [MaskedLMOutput] (/docs/transformers/v5.14.0/en/main_classes/output#transformers.modeling_outputs.MaskedLMOutput) or a tuple of
`torch.FloatTensor` (if `return_dict=False` is passed or when `config.return_dict=False`) comprising various
elements depending on the configuration ([RobertaConfig] (/docs/transformers/v5.14.0/en/model_doc/roberta#transformers.RobertaConfig)) and inputs.
The [RobertaForMaskedLM] (/docs/transformers/v5.14.0/en/model_doc/roberta#transformers.RobertaForMaskedLM) forward method, overrides the `__call__` special method.

Although the recipe for forward pass needs to be defined within this function, one should call the `Module`
instance afterwards instead of this since the former takes care of running the pre and post processing steps while
the latter silently ignores them.

- **loss** (`torch.FloatTensor` of shape `(1,)`, *optional*, returned when `labels` is provided) -- Masked language modeling (MLM) loss.
- **logits** (`torch.FloatTensor` of shape `(batch_size, sequence_length, config.vocab_size)`) -- Prediction scores of the language modeling head (scores for each vocabulary token before SoftMax).
- **hidden_states** (`tuple(torch.FloatTensor)`, *optional*, returned when `output_hidden_states=True` is passed or when `config.output_hidden_states=True`) -- Tuple of `torch.FloatTensor` (one for the output of the embeddings, if the model has an embedding layer, +
  one for the output of each layer) of shape `(batch_size, sequence_length, hidden_size)`.

  Hidden-states of the model at the output of each layer plus the optional initial embedding outputs.
- **attentions** (`tuple(torch.FloatTensor)`, *optional*, returned when `output_attentions=True` is passed or when `config.output_attentions=True`) -- Tuple of `torch.FloatTensor` (one for each layer) of shape `(batch_size, num_heads, sequence_length,
  sequence_length)`.

  Attentions weights after the attention softmax, used to compute the weighted average in the self-attention
  heads.

Example:

```python
>>> from transformers import AutoTokenizer, RobertaForMaskedLM
>>> import torch

>>> tokenizer = AutoTokenizer.from_pretrained("FacebookAI/roberta-base")
>>> model = RobertaForMaskedLM.from_pretrained("FacebookAI/roberta-base")

>>> inputs = tokenizer("The capital of France is <mask>.", return_tensors="pt")

>>> with torch.no_grad():
...     logits = model(**inputs).logits

>>> # retrieve index of <mask>
>>> mask_token_index = (inputs.input_ids == tokenizer.mask_token_id)
[0].nonzero(as_tuple=True)[0]

>>> predicted_token_id = logits[0, mask_token_index].argmax(axis=-1)
>>> tokenizer.decode(predicted_token_id)
...

>>> labels = tokenizer("The capital of France is Paris.", return_tensors="pt")
["input_ids"]
```

```
>>> # mask labels of non-<mask> tokens
>>> labels = torch.where(inputs.input_ids == tokenizer.mask_token_id, labels,
-100)

>>> outputs = model(**inputs, labels=labels)
>>> round(outputs.loss.item(), 2)
...
```

RobertaForSequenceClassification[[transformers.RobertaForSequenceClassification]]

- **config** ([RobertaForSequenceClassification] (/docs/transformers/v5.14.0/en/model_doc/roberta#transformers.RobertaForSequenceClassification)) --
  Model configuration class with all the parameters of the model. Initializing with a config file does not
  load the weights associated with the model, only the configuration. Check out the
  [from_pretrained()] (/docs/transformers/v5.14.0/en/main_classes/model#transformers.PreTrainedModel.from_pretrained) method to load the model weights.

RoBERTa Model transformer with a sequence classification/regression head on top (a linear layer on top of the
pooled output) e.g. for GLUE tasks.

This model inherits from [PreTrainedModel] (/docs/transformers/v5.14.0/en/main_classes/model#transformers.PreTrainedModel). Check the superclass documentation for the generic methods the
library implements for all its model (such as downloading or saving, resizing the input embeddings, pruning heads
etc.)

This model is also a PyTorch [torch.nn.Module] (https://pytorch.org/docs/stable/nn.html#torch.nn.Module) subclass.
Use it as a regular PyTorch Module and refer to the PyTorch documentation for all matter related to general usage
and behavior.

- **input_ids** (`torch.LongTensor` of shape `(batch_size, sequence_length)`, *optional*) --
  Indices of input sequence tokens in the vocabulary. Padding will be ignored by default.

  Indices can be obtained using [AutoTokenizer] (/docs/transformers/v5.14.0/en/model_doc/auto#transformers.AutoTokenizer). See [PreTrainedTokenizer.encode()]

(/docs/transformers/v5.14.0/en/internal/tokenization_utils#transformers.PreTrainedTokenizerBase.encode) and
  [PreTrainedTokenizer.__call__()](/docs/transformers/v5.14.0/en/internal/tokenization_utils#transformers.PreTrainedTokenizerBase.__call__) for details.

  [What are input IDs?](../glossary#input-ids)
- **attention_mask** (`torch.FloatTensor` of shape `(batch_size, sequence_length)`, *optional*) --
  Mask to avoid performing attention on padding token indices. Mask values selected in `[0, 1]`:

  - 1 for tokens that are **not masked**,
  - 0 for tokens that are **masked**.

  [What are attention masks?](../glossary#attention-mask)
- **token_type_ids** (`torch.LongTensor` of shape `(batch_size, sequence_length)`, *optional*) --
  Segment token indices to indicate first and second portions of the inputs. Indices are selected in `[0,1]`:

  - 0 corresponds to a *sentence A* token,
  - 1 corresponds to a *sentence B* token.
  This parameter can only be used when the model is initialized with `type_vocab_size` parameter with value
  >= 2. All the value in this tensor should be always < type_vocab_size.

  [What are token type IDs?](../glossary#token-type-ids)
- **position_ids** (`torch.LongTensor` of shape `(batch_size, sequence_length)`, *optional*) --
  Indices of positions of each input sequence tokens in the position embeddings. Selected in the range `[0, config.n_positions - 1]`.

  [What are position IDs?](../glossary#position-ids)
- **inputs_embeds** (`torch.FloatTensor` of shape `(batch_size, sequence_length, hidden_size)`, *optional*) --
  Optionally, instead of passing `input_ids` you can choose to directly pass an embedded representation. This
  is useful if you want more control over how to convert `input_ids` indices into associated vectors than the
  model's internal embedding lookup matrix.
- **labels** (`torch.LongTensor` of shape `(batch_size,)`, *optional*) --
  Labels for computing the sequence classification/regression loss. Indices should be in `[0, ...,
  config.num_labels - 1]`. If `config.num_labels == 1` a regression loss is computed (Mean-Square loss), If
  `config.num_labels > 1` a classification loss is computed (Cross-Entropy).
[SequenceClassifierOutput]

(/docs/transformers/v5.14.0/en/main_classes/output#transformers.modeling_outputs.SequenceClassifierOutput) or `tuple(torch.FloatTensor)`A [SequenceClassifierOutput] (/docs/transformers/v5.14.0/en/main_classes/output#transformers.modeling_outputs.SequenceClassifierOutput) or a tuple of
`torch.FloatTensor` (if `return_dict=False` is passed or when
`config.return_dict=False`) comprising various
elements depending on the configuration ([RobertaConfig]
(/docs/transformers/v5.14.0/en/model_doc/roberta#transformers.RobertaConfig)) and
inputs.
The [RobertaForSequenceClassification]
(/docs/transformers/v5.14.0/en/model_doc/roberta#transformers.RobertaForSequenceClassification) forward method, overrides the `__call__` special method.

Although the recipe for forward pass needs to be defined within this function, one
should call the `Module`
instance afterwards instead of this since the former takes care of running the pre
and post processing steps while
the latter silently ignores them.

- **loss** (`torch.FloatTensor` of shape `(1,)`, *optional*, returned when
`labels` is provided) -- Classification (or regression if config.num_labels==1)
loss.
- **logits** (`torch.FloatTensor` of shape `(batch_size, config.num_labels)`) --
Classification (or regression if config.num_labels==1) scores (before SoftMax).
- **hidden_states** (`tuple(torch.FloatTensor)`, *optional*, returned when
`output_hidden_states=True` is passed or when `config.output_hidden_states=True`)
-- Tuple of `torch.FloatTensor` (one for the output of the embeddings, if the
model has an embedding layer, +
  one for the output of each layer) of shape `(batch_size, sequence_length,
hidden_size)`.

  Hidden-states of the model at the output of each layer plus the optional initial
embedding outputs.
- **attentions** (`tuple(torch.FloatTensor)`, *optional*, returned when
`output_attentions=True` is passed or when `config.output_attentions=True`) --
Tuple of `torch.FloatTensor` (one for each layer) of shape `(batch_size,
num_heads, sequence_length,
  sequence_length)`.

  Attentions weights after the attention softmax, used to compute the weighted
average in the self-attention
  heads.

Example of single-label classification:

```python
>>> import torch
>>> from transformers import AutoTokenizer, RobertaForSequenceClassification
```

```
>>> tokenizer = AutoTokenizer.from_pretrained("FacebookAI/roberta-base")
>>> model = RobertaForSequenceClassification.from_pretrained("FacebookAI/roberta-
base")

>>> inputs = tokenizer("Hello, my dog is cute", return_tensors="pt")

>>> with torch.no_grad():
...     logits = model(**inputs).logits

>>> predicted_class_id = logits.argmax().item()
>>> model.config.id2label[predicted_class_id]
...

>>> # To train a model on `num_labels` classes, you can pass
`num_labels=num_labels` to `.from_pretrained(...)`
>>> num_labels = len(model.config.id2label)
>>> model = RobertaForSequenceClassification.from_pretrained("FacebookAI/roberta-
base", num_labels=num_labels)

>>> labels = torch.tensor([1])
>>> loss = model(**inputs, labels=labels).loss
>>> round(loss.item(), 2)
...
```

Example of multi-label classification:

```python
>>> import torch
>>> from transformers import AutoTokenizer, RobertaForSequenceClassification

>>> tokenizer = AutoTokenizer.from_pretrained("FacebookAI/roberta-base")
>>> model = RobertaForSequenceClassification.from_pretrained("FacebookAI/roberta-
base", problem_type="multi_label_classification")

>>> inputs = tokenizer("Hello, my dog is cute", return_tensors="pt")

>>> with torch.no_grad():
...     logits = model(**inputs).logits

>>> predicted_class_ids = torch.arange(0, logits.shape[-1])
[torch.sigmoid(logits).squeeze(dim=0) > 0.5]

>>> # To train a model on `num_labels` classes, you can pass
`num_labels=num_labels` to `.from_pretrained(...)`
>>> num_labels = len(model.config.id2label)
>>> model = RobertaForSequenceClassification.from_pretrained(
```

```
...     "FacebookAI/roberta-base", num_labels=num_labels,
problem_type="multi_label_classification"
... )

>>> labels = torch.sum(
...     torch.nn.functional.one_hot(predicted_class_ids[None, :].clone(),
num_classes=num_labels), dim=1
... ).to(torch.float)
>>> loss = model(**inputs, labels=labels).loss
```

RobertaForMultipleChoice[[transformers.RobertaForMultipleChoice]]

- **config** ([RobertaForMultipleChoice]
(/docs/transformers/v5.14.0/en/model_doc/roberta#transformers.RobertaForMultipleCh
oice)) --
  Model configuration class with all the parameters of the model. Initializing
with a config file does not
  load the weights associated with the model, only the configuration. Check out
the
  [from_pretrained()]
(/docs/transformers/v5.14.0/en/main_classes/model#transformers.PreTrainedModel.fro
m_pretrained) method to load the model weights.

The Roberta Model with a multiple choice classification head on top (a linear
layer on top of the pooled output and a
softmax) e.g. for RocStories/SWAG tasks.

This model inherits from [PreTrainedModel]
(/docs/transformers/v5.14.0/en/main_classes/model#transformers.PreTrainedModel).
Check the superclass documentation for the generic methods the
library implements for all its model (such as downloading or saving, resizing the
input embeddings, pruning heads
etc.)

This model is also a PyTorch [torch.nn.Module]
(https://pytorch.org/docs/stable/nn.html#torch.nn.Module) subclass.
Use it as a regular PyTorch Module and refer to the PyTorch documentation for all
matter related to general usage
and behavior.

- **input_ids** (`torch.LongTensor` of shape `(batch_size, num_choices,
sequence_length)`) --
  Indices of input sequence tokens in the vocabulary.

  Indices can be obtained using [AutoTokenizer]
(/docs/transformers/v5.14.0/en/model_doc/auto#transformers.AutoTokenizer). See
[PreTrainedTokenizer.encode()]

(/docs/transformers/v5.14.0/en/internal/tokenization_utils#transformers.PreTrainedTokenizerBase.encode) and
  [PreTrainedTokenizer.__call__()]
(/docs/transformers/v5.14.0/en/internal/tokenization_utils#transformers.PreTrainedTokenizerBase.__call__) for details.

  [What are input IDs?](../glossary#input-ids)
- **token_type_ids** (`torch.LongTensor` of shape `(batch_size, num_choices, sequence_length)`, *optional*) --
  Segment token indices to indicate first and second portions of the inputs. Indices are selected in `[0,1]`:

  - 0 corresponds to a *sentence A* token,
  - 1 corresponds to a *sentence B* token.
  This parameter can only be used when the model is initialized with `type_vocab_size` parameter with value
  >= 2. All the value in this tensor should be always < type_vocab_size.

  [What are token type IDs?](../glossary#token-type-ids)
- **attention_mask** (`torch.FloatTensor` of shape `(batch_size, sequence_length)`, *optional*) --
  Mask to avoid performing attention on padding token indices. Mask values selected in `[0, 1]`:

  - 1 for tokens that are **not masked**,
  - 0 for tokens that are **masked**.

  [What are attention masks?](../glossary#attention-mask)
- **labels** (`torch.LongTensor` of shape `(batch_size,)`, *optional*) --
  Labels for computing the multiple choice classification loss. Indices should be in `[0, ...,
  num_choices-1]` where `num_choices` is the size of the second dimension of the input tensors. (See
  `input_ids` above)
- **position_ids** (`torch.LongTensor` of shape `(batch_size, num_choices, sequence_length)`, *optional*) --
  Indices of positions of each input sequence tokens in the position embeddings. Selected in the range `[0,
  config.max_position_embeddings - 1]`.

  [What are position IDs?](../glossary#position-ids)
- **inputs_embeds** (`torch.FloatTensor` of shape `(batch_size, num_choices, sequence_length, hidden_size)`, *optional*) --
  Optionally, instead of passing `input_ids` you can choose to directly pass an embedded representation. This
  is useful if you want more control over how to convert `input_ids` indices into associated vectors than the
  model's internal embedding lookup matrix.[MultipleChoiceModelOutput]

(/docs/transformers/v5.14.0/en/main_classes/output#transformers.modeling_outputs.MultipleChoiceModelOutput) or `tuple(torch.FloatTensor)`A [MultipleChoiceModelOutput] (/docs/transformers/v5.14.0/en/main_classes/output#transformers.modeling_outputs.MultipleChoiceModelOutput) or a tuple of `torch.FloatTensor` (if `return_dict=False` is passed or when `config.return_dict=False`) comprising various elements depending on the configuration ([RobertaConfig] (/docs/transformers/v5.14.0/en/model_doc/roberta#transformers.RobertaConfig)) and inputs.
The [RobertaForMultipleChoice] (/docs/transformers/v5.14.0/en/model_doc/roberta#transformers.RobertaForMultipleChoice) forward method, overrides the `__call__` special method.

Although the recipe for forward pass needs to be defined within this function, one should call the `Module` instance afterwards instead of this since the former takes care of running the pre and post processing steps while the latter silently ignores them.

- **loss** (`torch.FloatTensor` of shape *(1,)*, *optional*, returned when `labels` is provided) -- Classification loss.
- **logits** (`torch.FloatTensor` of shape `(batch_size, num_choices)`) -- *num_choices* is the second dimension of the input tensors. (see *input_ids* above).

  Classification scores (before SoftMax).
- **hidden_states** (`tuple(torch.FloatTensor)`, *optional*, returned when `output_hidden_states=True` is passed or when `config.output_hidden_states=True`) -- Tuple of `torch.FloatTensor` (one for the output of the embeddings, if the model has an embedding layer, +
  one for the output of each layer) of shape `(batch_size, sequence_length, hidden_size)`.

  Hidden-states of the model at the output of each layer plus the optional initial embedding outputs.
- **attentions** (`tuple(torch.FloatTensor)`, *optional*, returned when `output_attentions=True` is passed or when `config.output_attentions=True`) -- Tuple of `torch.FloatTensor` (one for each layer) of shape `(batch_size, num_heads, sequence_length,
  sequence_length)`.

  Attentions weights after the attention softmax, used to compute the weighted average in the self-attention
  heads.

Example:

```python
>>> from transformers import AutoTokenizer, RobertaForMultipleChoice
>>> import torch

>>> tokenizer = AutoTokenizer.from_pretrained("FacebookAI/roberta-base")
>>> model = RobertaForMultipleChoice.from_pretrained("FacebookAI/roberta-base")

>>> prompt = "In Italy, pizza served in formal settings, such as at a restaurant,
is presented unsliced."
>>> choice0 = "It is eaten with a fork and a knife."
>>> choice1 = "It is eaten while held in the hand."
>>> labels = torch.tensor(0).unsqueeze(0)  # choice0 is correct (according to
Wikipedia ;)), batch size 1

>>> encoding = tokenizer([prompt, prompt], [choice0, choice1],
return_tensors="pt", padding=True)
>>> outputs = model(**{k: v.unsqueeze(0) for k, v in encoding.items()},
labels=labels)  # batch size is 1

>>> # the linear classifier still needs to be trained
>>> loss = outputs.loss
>>> logits = outputs.logits
```

RobertaForTokenClassification[[transformers.RobertaForTokenClassification]]

- **config** ([RobertaForTokenClassification]
(/docs/transformers/v5.14.0/en/model_doc/roberta#transformers.RobertaForTokenClass
ification)) --
  Model configuration class with all the parameters of the model. Initializing
with a config file does not
  load the weights associated with the model, only the configuration. Check out
the
  [from_pretrained()]
(/docs/transformers/v5.14.0/en/main_classes/model#transformers.PreTrainedModel.fro
m_pretrained) method to load the model weights.

The Roberta transformer with a token classification head on top (a linear layer on
top of the hidden-states
output) e.g. for Named-Entity-Recognition (NER) tasks.

This model inherits from [PreTrainedModel]
(/docs/transformers/v5.14.0/en/main_classes/model#transformers.PreTrainedModel).
Check the superclass documentation for the generic methods the
library implements for all its model (such as downloading or saving, resizing the
input embeddings, pruning heads
etc.)

This model is also a PyTorch [torch.nn.Module]
(https://pytorch.org/docs/stable/nn.html#torch.nn.Module) subclass.
Use it as a regular PyTorch Module and refer to the PyTorch documentation for all
matter related to general usage
and behavior.

- **input_ids** (`torch.LongTensor` of shape `(batch_size, sequence_length)`,
optional) --
  Indices of input sequence tokens in the vocabulary. Padding will be ignored by
default.

  Indices can be obtained using [AutoTokenizer]
(/docs/transformers/v5.14.0/en/model_doc/auto#transformers.AutoTokenizer). See
[PreTrainedTokenizer.encode()]
(/docs/transformers/v5.14.0/en/internal/tokenization_utils#transformers.PreTrained
TokenizerBase.encode) and
  [PreTrainedTokenizer.__call__()]
(/docs/transformers/v5.14.0/en/internal/tokenization_utils#transformers.PreTrained
TokenizerBase.__call__) for details.

  [What are input IDs?](../glossary#input-ids)
- **attention_mask** (`torch.FloatTensor` of shape `(batch_size,
sequence_length)`, *optional*) --
  Mask to avoid performing attention on padding token indices. Mask values
selected in `[0, 1]`:

  - 1 for tokens that are **not masked**,
  - 0 for tokens that are **masked**.

  [What are attention masks?](../glossary#attention-mask)
- **token_type_ids** (`torch.LongTensor` of shape `(batch_size, sequence_length)`,
optional) --
  Segment token indices to indicate first and second portions of the inputs.
Indices are selected in `[0,1]`:

  - 0 corresponds to a *sentence A* token,
  - 1 corresponds to a *sentence B* token.
  This parameter can only be used when the model is initialized with
`type_vocab_size` parameter with value
  >= 2. All the value in this tensor should be always < type_vocab_size.

  [What are token type IDs?](../glossary#token-type-ids)
- **position_ids** (`torch.LongTensor` of shape `(batch_size, sequence_length)`,
optional) --
  Indices of positions of each input sequence tokens in the position embeddings.
Selected in the range `[0, config.n_positions - 1]`.

  [What are position IDs?](../glossary#position-ids)

- **inputs_embeds** (`torch.FloatTensor` of shape `(batch_size, sequence_length, hidden_size)`, *optional*) --
  Optionally, instead of passing `input_ids` you can choose to directly pass an embedded representation. This
  is useful if you want more control over how to convert `input_ids` indices into associated vectors than the
  model's internal embedding lookup matrix.
- **labels** (`torch.LongTensor` of shape `(batch_size, sequence_length)`, *optional*) --
  Labels for computing the token classification loss. Indices should be in `[0, ..., config.num_labels - 1]`.[TokenClassifierOutput] (/docs/transformers/v5.14.0/en/main_classes/output#transformers.modeling_outputs.TokenClassifierOutput) or `tuple(torch.FloatTensor)`A [TokenClassifierOutput] (/docs/transformers/v5.14.0/en/main_classes/output#transformers.modeling_outputs.TokenClassifierOutput) or a tuple of
`torch.FloatTensor` (if `return_dict=False` is passed or when
`config.return_dict=False`) comprising various
elements depending on the configuration ([RobertaConfig] (/docs/transformers/v5.14.0/en/model_doc/roberta#transformers.RobertaConfig)) and inputs.
The [RobertaForTokenClassification] (/docs/transformers/v5.14.0/en/model_doc/roberta#transformers.RobertaForTokenClassification) forward method, overrides the `__call__` special method.

Although the recipe for forward pass needs to be defined within this function, one should call the `Module`
instance afterwards instead of this since the former takes care of running the pre and post processing steps while
the latter silently ignores them.

- **loss** (`torch.FloatTensor` of shape `(1,)`, *optional*, returned when `labels` is provided) -- Classification loss.
- **logits** (`torch.FloatTensor` of shape `(batch_size, sequence_length, config.num_labels)`) -- Classification scores (before SoftMax).
- **hidden_states** (`tuple(torch.FloatTensor)`, *optional*, returned when `output_hidden_states=True` is passed or when `config.output_hidden_states=True`) -- Tuple of `torch.FloatTensor` (one for the output of the embeddings, if the model has an embedding layer, +
  one for the output of each layer) of shape `(batch_size, sequence_length, hidden_size)`.

  Hidden-states of the model at the output of each layer plus the optional initial embedding outputs.
- **attentions** (`tuple(torch.FloatTensor)`, *optional*, returned when `output_attentions=True` is passed or when `config.output_attentions=True`) --
Tuple of `torch.FloatTensor` (one for each layer) of shape `(batch_size, num_heads, sequence_length,
  sequence_length)`.

  Attentions weights after the attention softmax, used to compute the weighted average in the self-attention
  heads.

Example:

```python
>>> from transformers import AutoTokenizer, RobertaForTokenClassification
>>> import torch

>>> tokenizer = AutoTokenizer.from_pretrained("FacebookAI/roberta-base")
>>> model = RobertaForTokenClassification.from_pretrained("FacebookAI/roberta-base")

>>> inputs = tokenizer(
...     "HuggingFace is a company based in Paris and New York", add_special_tokens=False, return_tensors="pt"
... )

>>> with torch.no_grad():
...     logits = model(**inputs).logits

>>> predicted_token_class_ids = logits.argmax(-1)

>>> # Note that tokens are classified rather then input words which means that
>>> # there might be more predicted token classes than words.
>>> # Multiple token classes might account for the same word
>>> predicted_tokens_classes = [model.config.id2label[t.item()] for t in predicted_token_class_ids[0]]
>>> predicted_tokens_classes
...

>>> labels = predicted_token_class_ids
>>> loss = model(**inputs, labels=labels).loss
>>> round(loss.item(), 2)
...
```

RobertaForQuestionAnswering[[transformers.RobertaForQuestionAnswering]]

- **config** ([RobertaForQuestionAnswering](/docs/transformers/v5.14.0/en/model_doc/roberta#transformers.RobertaForQuestionAnswering)) --
  Model configuration class with all the parameters of the model. Initializing with a config file does not
  load the weights associated with the model, only the configuration. Check out the

  [from_pretrained()]
(/docs/transformers/v5.14.0/en/main_classes/model#transformers.PreTrainedModel.fro
m_pretrained) method to load the model weights.

The Roberta transformer with a span classification head on top for extractive
question-answering tasks like
SQuAD (a linear layer on top of the hidden-states output to compute `span start
logits` and `span end logits`).

This model inherits from [PreTrainedModel]
(/docs/transformers/v5.14.0/en/main_classes/model#transformers.PreTrainedModel).
Check the superclass documentation for the generic methods the
library implements for all its model (such as downloading or saving, resizing the
input embeddings, pruning heads
etc.)

This model is also a PyTorch [torch.nn.Module]
(https://pytorch.org/docs/stable/nn.html#torch.nn.Module) subclass.
Use it as a regular PyTorch Module and refer to the PyTorch documentation for all
matter related to general usage
and behavior.

- **input_ids** (`torch.LongTensor` of shape `(batch_size, sequence_length)`,
optional) --
  Indices of input sequence tokens in the vocabulary. Padding will be ignored by
default.

  Indices can be obtained using [AutoTokenizer]
(/docs/transformers/v5.14.0/en/model_doc/auto#transformers.AutoTokenizer). See
[PreTrainedTokenizer.encode()]
(/docs/transformers/v5.14.0/en/internal/tokenization_utils#transformers.PreTrained
TokenizerBase.encode) and
  [PreTrainedTokenizer.__call__()]
(/docs/transformers/v5.14.0/en/internal/tokenization_utils#transformers.PreTrained
TokenizerBase.__call__) for details.

  [What are input IDs?](../glossary#input-ids)
- **attention_mask** (`torch.FloatTensor` of shape `(batch_size,
sequence_length)`, *optional*) --
  Mask to avoid performing attention on padding token indices. Mask values
selected in `[0, 1]`:

  - 1 for tokens that are **not masked**,
  - 0 for tokens that are **masked**.

  [What are attention masks?](../glossary#attention-mask)
- **token_type_ids** (`torch.LongTensor` of shape `(batch_size, sequence_length)`,
optional) --

  Segment token indices to indicate first and second portions of the inputs.
Indices are selected in `[0,1]`:

  - 0 corresponds to a *sentence A* token,
  - 1 corresponds to a *sentence B* token.
  This parameter can only be used when the model is initialized with
`type_vocab_size` parameter with value
  >= 2. All the value in this tensor should be always < type_vocab_size.

  [What are token type IDs?](../glossary#token-type-ids)
- **position_ids** (`torch.LongTensor` of shape `(batch_size, sequence_length)`,
optional) --
  Indices of positions of each input sequence tokens in the position embeddings.
Selected in the range `[0, config.n_positions - 1]`.

  [What are position IDs?](../glossary#position-ids)
- **inputs_embeds** (`torch.FloatTensor` of shape `(batch_size, sequence_length,
hidden_size)`, *optional*) --
  Optionally, instead of passing `input_ids` you can choose to directly pass an
embedded representation. This
  is useful if you want more control over how to convert `input_ids` indices into
associated vectors than the
  model's internal embedding lookup matrix.
- **start_positions** (`torch.LongTensor` of shape `(batch_size,)`, *optional*) --
  Labels for position (index) of the start of the labelled span for computing the
token classification loss.
  Positions are clamped to the length of the sequence (`sequence_length`).
Position outside of the sequence
  are not taken into account for computing the loss.
- **end_positions** (`torch.LongTensor` of shape `(batch_size,)`, *optional*) --
  Labels for position (index) of the end of the labelled span for computing the
token classification loss.
  Positions are clamped to the length of the sequence (`sequence_length`).
Position outside of the sequence
  are not taken into account for computing the loss.[QuestionAnsweringModelOutput]
(/docs/transformers/v5.14.0/en/main_classes/output#transformers.modeling_outputs.Q
uestionAnsweringModelOutput) or `tuple(torch.FloatTensor)`A
[QuestionAnsweringModelOutput]
(/docs/transformers/v5.14.0/en/main_classes/output#transformers.modeling_outputs.Q
uestionAnsweringModelOutput) or a tuple of
`torch.FloatTensor` (if `return_dict=False` is passed or when
`config.return_dict=False`) comprising various
elements depending on the configuration ([RobertaConfig]
(/docs/transformers/v5.14.0/en/model_doc/roberta#transformers.RobertaConfig)) and
inputs.
The [RobertaForQuestionAnswering]
(/docs/transformers/v5.14.0/en/model_doc/roberta#transformers.RobertaForQuestionAn
swering) forward method, overrides the `__call__` special method.

Although the recipe for forward pass needs to be defined within this function, one should call the `Module`
instance afterwards instead of this since the former takes care of running the pre and post processing steps while
the latter silently ignores them.

- **loss** (`torch.FloatTensor` of shape `(1,)`, *optional*, returned when `labels` is provided) -- Total span extraction loss is the sum of a Cross-Entropy for the start and end positions.
- **start_logits** (`torch.FloatTensor` of shape `(batch_size, sequence_length)`) -- Span-start scores (before SoftMax).
- **end_logits** (`torch.FloatTensor` of shape `(batch_size, sequence_length)`) -- Span-end scores (before SoftMax).
- **hidden_states** (`tuple(torch.FloatTensor)`, *optional*, returned when `output_hidden_states=True` is passed or when `config.output_hidden_states=True`) -- Tuple of `torch.FloatTensor` (one for the output of the embeddings, if the model has an embedding layer, +
  one for the output of each layer) of shape `(batch_size, sequence_length, hidden_size)`.

  Hidden-states of the model at the output of each layer plus the optional initial embedding outputs.
- **attentions** (`tuple(torch.FloatTensor)`, *optional*, returned when `output_attentions=True` is passed or when `config.output_attentions=True`) -- Tuple of `torch.FloatTensor` (one for each layer) of shape `(batch_size, num_heads, sequence_length,
  sequence_length)`.

  Attentions weights after the attention softmax, used to compute the weighted average in the self-attention
  heads.

Example:

```python
>>> from transformers import AutoTokenizer, RobertaForQuestionAnswering
>>> import torch

>>> tokenizer = AutoTokenizer.from_pretrained("FacebookAI/roberta-base")
>>> model = RobertaForQuestionAnswering.from_pretrained("FacebookAI/roberta-base")

>>> question, text = "Who was Jim Henson?", "Jim Henson was a nice puppet"

>>> inputs = tokenizer(question, text, return_tensors="pt")
>>> with torch.no_grad():
...     outputs = model(**inputs)
```

```
>>> answer_start_index = outputs.start_logits.argmax()
>>> answer_end_index = outputs.end_logits.argmax()

>>> predict_answer_tokens = inputs.input_ids[0, answer_start_index :
answer_end_index + 1]
>>> tokenizer.decode(predict_answer_tokens, skip_special_tokens=True)
...

>>> # target is "nice puppet"
>>> target_start_index = torch.tensor([14])
>>> target_end_index = torch.tensor([15])

>>> outputs = model(**inputs, start_positions=target_start_index,
end_positions=target_end_index)
>>> loss = outputs.loss
>>> round(loss.item(), 2)
...
```